Memcached vs Redis

a simple fast cache, or a data structure server

Watch the 4-min video

In one line: Memcached is a deliberately simple, multithreaded, volatile key-value cache with slab memory and client-side sharding; Redis is a data structure server with atomic operations, Lua, persistence, replication and pub/sub; pick Memcached for a pure blob cache on big multi-core boxes, Redis for everything else, which is most new systems.

1 Is Memcached just the old choice?

Everyone says Redis, and Redis is great. But Facebook, Wikipedia and many large sites have run Memcached for years, at enormous scale. Is it just the old choice, or sometimes the better one?

2 Memcached: deliberately simple

Memcached is an in-memory key-value store: you `get`, `set`, `delete` and `incr` **opaque values** by key, with a TTL. No data structures, no persistence, no replication, no scripting. Clients spread keys across servers with consistent hashing (ketama, lesson 5), so the servers don't even know about each other. Values are limited to 1 MB by default.

3 Threads and memory

Memcached is **multithreaded**: worker threads handle connections in parallel, so a single instance uses a big machine fully. Redis runs commands on one thread (lesson 27), so to use many cores you usually run several Redis instances or a cluster.

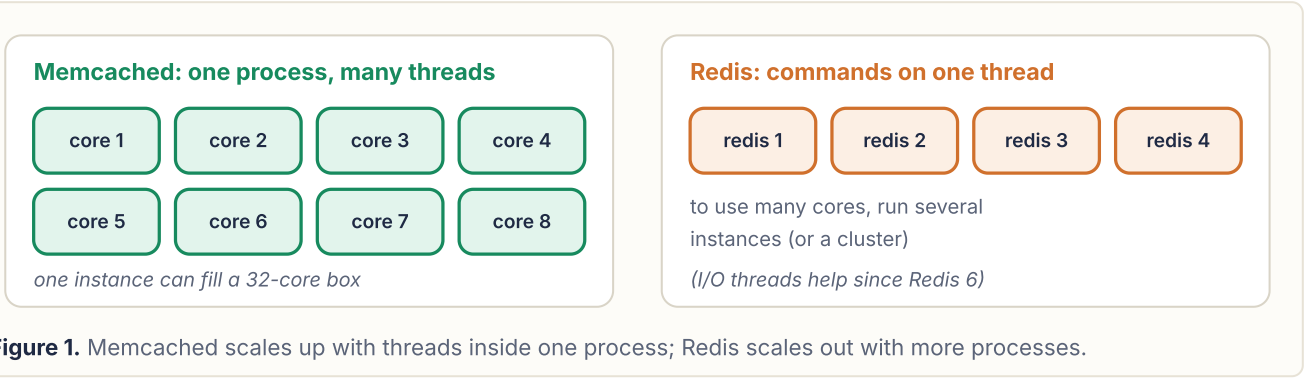


Figure 1. Memcached scales up with threads inside one process; Redis scales out with more processes.

Memory management differs too. Memcached uses a **slab allocator**: memory is divided into slab classes of fixed-size chunks (64 B, 128 B, 256 B, ... growing by a factor of 1.25 by default). Each item goes into the smallest chunk that fits. There's no fragmentation over time, and allocation is very fast. The cost is wasted space inside chunks, and if your item sizes shift, memory may be stuck in the wrong slab class.

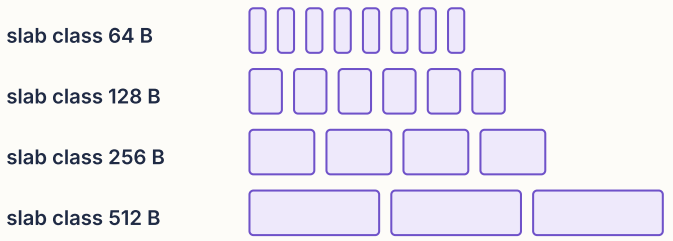


Figure 2. Slab classes of fixed-size chunks (simplified): each item takes the smallest chunk that fits.

4 Redis: a data structure server

Hashes, lists, sets, sorted sets, streams, pub/sub, Lua scripts, transactions, persistence with RDB and AOF, replication, Sentinel and Cluster (lessons 27 and 28). Redis can be a cache, a rate limiter, a queue, a leaderboard, a lock service, or a primary store for some use cases.

	Memcached	Redis
Data model	opaque strings / blobs	strings, hashes, lists, sets, sorted sets, streams
Threads	multithreaded	single-threaded commands (I/O threads optional)
Persistence	none	RDB snapshots, AOF
Replication / HA	none built in (client-side)	replicas, Sentinel, Cluster
Scaling out	client-side consistent hashing	Redis Cluster hash slots, or client/proxy sharding
Atomic logic	incr, CAS	every command, Lua scripts, transactions
Memory	slab allocator	jemalloc, can fragment (active defrag helps)

WORKED EXAMPLE: HTML FRAGMENT CACHE

A content site renders HTML fragments (a product card, a sidebar) and caches each one as a string for 5 minutes. That's pure key-value: get or set a blob, nothing else. Memcached fits perfectly: multithreading uses all the cores on a large cache box, slab allocation handles millions of similar-sized fragments, and losing the cache on restart is fine, because fragments can be rebuilt.

5 So which one?

- **Memcached**: a simple, volatile key-value cache of strings or serialised objects, at very high throughput, using many cores on big nodes, with minimal operational features.
- **Redis**: when you need data structures, atomic operations, persistence, replication and failover, pub/sub, or one tool that covers caching and more.

For most new systems Redis (or a compatible fork such as Valkey) is the default, and that's fine. Saying *why* is what earns points in an interview.

COMMON MISTAKES

- Storing data you can't rebuild in Memcached: it has no persistence.
- Running one Redis instance on a 64-core box and calling it saturated at one core.
- Picking Redis then using it only for get/set, and paying for persistence you don't need (turn it off for a pure cache).

6 Key terms

Opaque value: the cache doesn't look inside it

Slab class: one chunk size (64 B, 128 B, ...)

CAS: check-and-set: update only if unchanged

Slab allocator: fixed-size chunk classes, no fragmentation

Client-side sharding: clients pick the server by hashing

Valkey: open-source Redis fork

7 Interview questions

Q1. When would you pick Memcached over Redis? BASIC

For a pure, volatile key-value cache of strings or blobs, like HTML fragments or serialised objects, where you want multithreaded performance on large machines, predictable slab memory, and no need for data structures, persistence or replication.

Q2. What does Redis add? BASIC

Rich data structures, atomic operations and Lua, persistence, replication with failover, and pub/sub.

Q3. You need a leaderboard, a rate limiter and a page cache. One tool or two? SCENARIO

Redis for all three: sorted sets for the leaderboard, INCR plus EXPIRE in Lua for the limiter, strings with TTLs for pages. Memcached can't do the first two.

Q4. Memcached evicts items even though stats show free memory. Why? SCENARIO

Slab calcification: memory is assigned to slab classes for old item sizes, and the class your new items need is full. Enable the slab rebalancer/automove, or restart with tuned growth factors.

Q5. How does Memcached scale across servers without a cluster protocol? TRICKY

Clients hash keys onto servers with consistent hashing, so each client independently finds the right server; all clients must share the same server list and hash function.

Q6. Is Redis single-threaded a weakness? TRICKY

Rarely for caching: it's usually network-bound, commands are fast, and you scale out with more instances. It matters for CPU-heavy commands and for using a big multi-core box with one instance.

Next: SD-F-016 · Distributed Caching backendroadmap.com